LEARNING TO PICK AND PLACE JENGA BLOCKS

A π0.5-Based Approach for the Elite CS66 Robot

Presented by: Mohd Arhan

Gmail- mohaarhan686@gmail.com

Focus: Problem Definition · Robot Learning · Evaluation · Project Execution

Proposed approach

Fine-tune the open-source π0.5 policy, validate it through controlled robot trials, and retain a verified fallback.

Part A

Problem Definition & Success Criteria

What are we trying to achieve?

The aim is to fine-tune the open-source π0.5 model so that the Elite Robots CS66 arm, fitted with a Robotiq 2F-85 gripper, can pick one available Jenga block and place it according to the given instruction.

There are two target locations:

- Inside a marked boundary on the table.
- On top of an existing stack without knocking it over.

My initial approach

Start with accessible blocks and a fixed, controlled workspace. Once the basic task works reliably, test changes in block positions, orientations, stack height and lighting.

These targets are initial proposals. I would confirm the acceptance criteria with the company before starting the experiments.

How will I measure success?

METRIC	PROPOSED V1 TARGET
Overall task success	⩾80%
Success in each goal type	⩾75%
Stack remains standing	⩾95%
Median completion time	⩽35 seconds
Maximum episode time	60 seconds

Generalisation & Initial Assumptions

– *What should the robot handle in v1?*

- **Different block positions and orientations.**
- **Different numbers of accessible blocks.**
- **Both placement goals and several stack heights.**
- **Small lighting changes and minor camera shifts.**
- **Limited, predefined distractor objects.**

– What will I leave for later?

- Blocks that are deeply wedged or inaccessible.
- Moving blocks or a moving table.
- Recovery after a toppled stack.
- Major changes in camera arrangement or workspace.
- Unusual objects that are not part of the Jenga task.

My assumptions

I am assuming that the robot will work in a **fixed tabletop workspace,** with **standard wooden Jenga blocks** and a **stationary stack**. The robot can select any accessible block, and the two placement goals will be given as clear language instructions.

I am also assuming that the company will provide access to the robot, gripper, cameras, pretrained model and the compute needed for fine-tuning. The exact interfaces and compatibility still need to be checked.

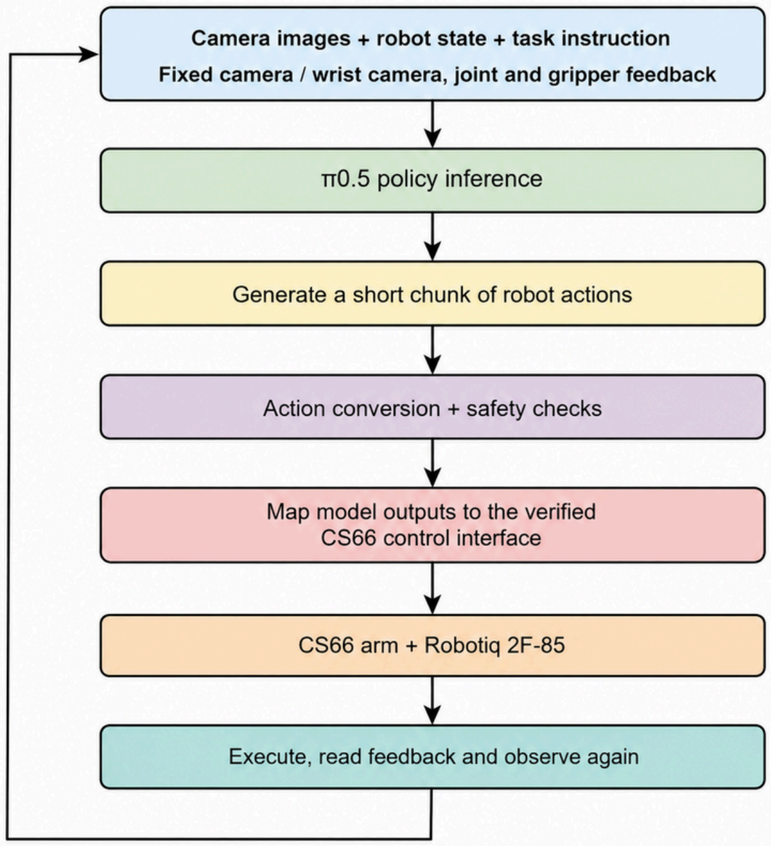
– Three things I need to clarify before starting

1. What is the exact camera setup, and what observation format can π0.5 receive?
2. What are the precise placement and stack-stability acceptance rules?
3. What robot-control interface, compute resources and safety limits are available for real-world training and deployment?

I would confirm these points before locking the data-collection plan and release targets.

Part B — Proposed Technical Approach

1. System architecture



Proposed closed loop. Camera arrangement, action format and control frequency must be confirmed before implementation.

2. Data strategy

- Start with **100–200 successful demonstrations** as a pilot target; expand based on validation.
- Use **supervised teleoperation** for both placement goals; **record synchronized images**, robot state, gripper state, instructions and actions.
- Vary block positions, orientations, accessible-block choices and stack heights; **remove failed or incorrectly synchronized recordings** from the clean training set.

3. Training strategy

- **Adapt pretrained π0.5** using the official OpenPI pipeline and a robot-specific dataset configuration.
-
- Start with **parameter-efficient fine-tuning** (LoRA, if supported by the selected configuration); **compare checkpoints** on validation trials.
-
- **Train on an NVIDIA GPU**; budget initially for about 24 GB+ VRAM for LoRA, subject to the exact model/configuration. **Keep inference separate** from robot safety control.

Part B

— *Validation & Deployment Plan*

4. *Evaluation protocol*

- Use repeatable trials for both goals; report success separately and overall.
- Begin with 50 held-out trials per goal as an initial evaluation proposal; expand the sample if results are inconsistent.
- Track task success, stack stability, completion time, placement accuracy, timeout rate and safety stops.
- Keep final test layouts and demonstration sessions separate from training; test unseen positions, orientations and validated lighting changes.

5. *Baselines & fallbacks*

- Compare π0.5 against teleoperated execution and a simple scripted pick-and-place baseline in the same workspace.
- If fine-tuning underperforms: check camera/state/action mapping, data quality and latency; add targeted demonstrations and retrain.
- If needed, restrict v1 to validated block layouts and use supervised teleoperation or scripted motion as the fallback.

6. *Safety*

- Keep a reachable emergency stop, trained human supervision and low-speed initial tests.
- Configure verified joint, TCP, workspace and collision limits; use a clear exclusion zone around the robot.
- Set gripper force and closing limits within the approved setup; stop on unexpected contact, loss of control or unstable stacking.
- Enforce limits outside the learned policy; never rely on π0.5 alone for safety.

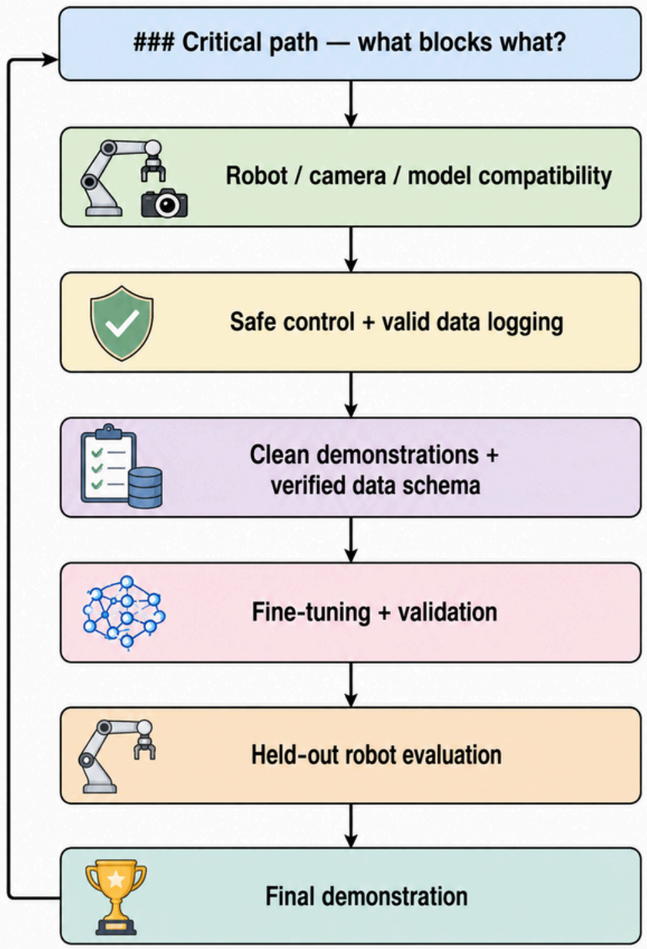
Phased Roadmap and Critical Path

Part C — Proposed 10-week implementation plan | 5-person team | Subject to hardware, model and compute access

From compatibility checks to a validated robot demonstration

Weeks are planning estimates, not completed work. Each phase has a measurable exit gate.

Weeks	Phase / main work	Deliverable and go/no-go gate
1-2	**1. Compatibility & safety** *Verify CS66, 2F-85, cameras, OpenPI setup, control interface and GPU.*	Documented input/output mapping; camera and state data received; supervised low-speed motion and stop procedure verified. Gate: **do not collect demonstrations until the full interface works.**
3	***2. Baseline & recording pipeline*** *Establish simple pick-and-place and synchronized logging.*	Repeatable baseline trial; inspect image/state/action timestamps and gripper feedback. **Gate: fix control or recording errors before dataset collection.**
4-5	***3. Demonstrations & data validation*** *Collect pilot data for both placement goals.*	Target 100–200 clean demonstrations; validate data schema and train/validation split. **Gate: data must match the model's required fields and action format**.
6-7	***4. Fine-tune π0.5*** *Run a small training experiment, then compare checkpoints.*	Reproducible training run and validation report. **Gate: proceed only if training is stable and validation justifies a controlled robot test.**
8–9	***5. Robot evaluation*** *Test both goals, held-out layouts, latency and safety behaviour.*	Proposed initial test: 50 trials per goal; report success, stability, time, placement accuracy and safety stops. **Gate: compare with baseline and investigate failures.**
10	***6. Final demo & handover*** *Repeat validated task; prepare results and fallback.*	Recorded demonstration, test summary, known limitations and reproducible setup. **Gate: present only verified capabilities; retain supervised/scripted fallback**.



Top Risks, Early Detection and Mitigation

Risk Plan — Detect Blockers Before They Delay the Demo

Prioritise risks that can block the critical path. These are anticipated risks, not reported failures.

Risk	Early warning / detection	Mitigation / fallback
1. Robot–model incompatibility	Camera/state fields do not match π0.5 inputs; action dimensions, units or control interface are unclear	**Build a minimal input/output test first;** verify camera mapping, coordinate frames and controller commands. If blocked, use scripted control while resolving compatibility.
2. Poor demonstrations	Missing timestamps, inconsistent actions, incorrect gripper labels or failures concentrated in one goal.	**Inspect sample recordings before scaling** up; fix synchronisation and logging. Collect targeted demonstrations for under-covered cases.
3. Fine-tuning / GPU failure	Out-of-memory errors, unstable training, unavailable compatible GPU or no validation improvement.	**Run a small training smoke test early;** verify the supported backend and memory requirements. Try a documented lower-memory setup if available; otherwise use approved compute or the fallback baseline.
5. Unreliable evaluation	Success rules change between trials; test layouts overlap training data; results cannot be reproduced.	**Freeze pass/fail rules**; separate training, validation and held-out tests; log all valid attempts, failures, timeouts and safety stops.
6. Hardware / safety incident	Unexpected motion, collisions, slipping blocks, unstable stacks or unverified stop/limit behaviour.	**Verify approved safety procedure and limits before autonomy**; start at low speed with supervision. Stop, inspect and correct the cause before resuming.
7. Schedule or access delay	Robot, cameras, compute or operator access not confirmed by Week 1–2	Escalate early; **do offline setup and data-schema** work in parallel. Protect the final evaluation window; if blocked, present a clearly labelled partial result rather than claim an untested autonomous demo.

Critical blocker: if robot commands, model inputs/outputs or data recording do not work, later training and evaluation cannot be trusted. ***Dataset preparation and safety documentation can proceed in parallel***, but physical deployment must wait for verified control and safety.

Decision rule

Go: interface verified, safety checks passed, data valid and the next phase has a measurable test.

Pause / debug: the blocker is understood and a specific fix can be tested within the schedule.

Change approach: a core dependency remains unavailable or repeated tests show no credible path to the planned result; switch to a validated baseline and report the limitation.

Team Structure & Responsibility Matrix

Recommended team: 4 core contributors + optional part-time specialist support

Role	Seniority	Primary responsibility
Project / Robot-Learning Lead	Experienced engineer	Technical decisions, π0.5 adaptation, training strategy, milestone acceptanExperienced engineerce
Robot Integration Engineer	Mid-level engineer	CS66, Robotiq gripper, cameras, control interface, logging and deployment
Robotics / Mechanical Intern — Me	Intern / developing engineer	Setup support, data collection, mechanical checks, testing, documentation and coordination
Safety / Lab Supervisor	Authorised, experienced; part-time	Operating limits, risk review, emergency-stop verification and deployment approval

Compact RACI matrix

R = Responsible · A = Accountable · C = Consulted · I = Informed

Workstream	Learning Lead	Integration Engineer	Me	Safety Supervisor
Robot, gripper & camera integration	C	**A/R**	R	C
Demonstration collection & data checks	A	R	**R**	C
π0.5 fine-tuning & model configuration	**A/R**	C	I	I
Evaluation & results reporting	A	R	**R**	C
Safety review & deployment approval	C	R	I	**A**
Schedule, risks & coordination	**A**	C	R	I

Lean-team alternative:

3 people by combining project coordination with the learning lead, and combining data collection with integration support. Keep model-training accountability with someone competent in the selected OpenPI implementation; keep safety approval with an authorised, qualified person.

Working rule:

One accountable owner per workstream. Owners provide evidence at each gate; the project lead tracks the ***Part C milestones*** and escalates blockers early.

My Role, Team Management & Outsourcing

My contribution — Mechanical / Robotics Intern

- Support CS66–gripper setup, workspace checks and mechanical integration.
- Assist with camera placement, calibration checks and synchronised demonstration recording.
- Validate data completeness, timestamps, labels and train/test separation.
- Run approved test trials; log success, placement quality, timeouts and failures.
- Maintain test procedures, issue tracker, meeting notes and milestone evidence.
-

Boundaries & escalation

- I would support, not independently own, advanced π0.5 training infrastructure, unfamiliar controller changes or safety-critical deployment decisions.
- The learning lead owns model-training decisions; the integration engineer owns control-interface implementation; the authorised supervisor approves safety readiness.
- Stop and escalate unclear motion, unsafe behaviour, invalid data or blocked milestones.

Team operating rhythm

- **Daily:** short blocker check during active integration or data collection.
- **Weekly**: technical review, issue tracker, risks, next-week commitments.
- **At each gate**: demonstrate evidence against acceptance criteria before proceeding.
- **Leadership update**: completed work, measured results, blockers, risk/impact, resource request and next steps.

Quality verification

- Code/configuration review and reproducible run logs.
- Data-schema, timestamp, label and split checks.
- Baseline comparison and held-out robot trials.
- Written pass/fail criteria, failure logs and supervisor sign-off where required

.

Outsource selectively

- Possible: bounded data annotation or a short specialist consultation.
- Keep in-house: robot control, safety decisions, final evaluation and acceptance.
- Before outsourcing, agree on cost, sample checks, confidentiality, access controls, deliverables and acceptance criteria.

***Principle:* Delegate tasks, verify evidence, retain clear accountability. Use the Part C 10-week plan as the baseline, conditional on timely robot, compute and staff access.**